PROGRAMA DE CAPACITACIÓN · DESARROLLANDO CON IA

Scrum + SDD

Valentina y TurnoVet: una historia, de la idea al commit

Material de estudio

Tu modelo para el paso 4 del taller

LA DEV

Valentina

Desarrolladora web freelance. Trabaja sola, pero no improvisa.

EL CLIENTE

Patitas Felices

Veterinaria de barrio. Quiere ver avances cada semana.

EL PRODUCTO

TurnoVet

App web donde los clientes reservan horas de atención para sus mascotas.

Plazo: 1 mes

1 solo cliente

Avances cada semana

Cómo leerlo. Sigue una sola historia de usuario por todo el ciclo. Cuando llegues al paso 4 del taller, la página 9 te dice qué parte del caso mirar para cada fila de tu mapa.

Caso ficticio y educativo, sin relación con el proyecto integrador.

01 · LA IDEA

El qué y el cómo

> Imaginemos que un desarrollador trabaja solo, pero usa Scrum para organizar su proyecto y SDD para estructurar su desarrollo técnico.

Valentina sabe que sin orden va a terminar haciendo **vibe coding** y entregando algo que falla. Por eso combina dos herramientas que responden preguntas distintas.

Scrum

¿Qué hago esta semana y en qué orden?

NIVEL

Organización del trabajo

ARTEFACTOS

Product Backlog · Sprint Backlog · Incremento

+

SDD

¿Cómo construyo bien cada cosa?

NIVEL

Desarrollo técnico

ARTEFACTOS

spec.md · plan.md · tareas · tests

Scrum elige la historia correcta → **SDD la construye de la forma correcta**

SDD = Spec-Driven Development. Primero se escribe con precisión qué debe hacer el software y cómo se comprobará; recién después se genera el código. Lo profundizas esta tarde, en la Unidad 2.

Vibe coding es programar describiendo «la onda» y aceptar lo que salga sin leerlo. Sirve para explorar; para producción, no.

LA RUTA DE ESTE CASO

Scrum pág. 3 · Flujo completo pág. 4 · SDD: spec → plan → tests pág. 5 a 7 · Cierre pág. 8 · Tu mapa pág. 9

02 · SCRUM DE UNA SOLA PERSONA

Una dev, tres sombreros

Scrum está pensado para equipos, pero Valentina toma sus piezas y cumple los tres roles.

Product Owner
Prioriza con el cliente qué va primero.

Dev
Construye cada historia.

Scrum Master
Se cuida de no saltarse el proceso.

SU SEMANA · UN SPRINT DE 1 SEMANA

Evento · Artefacto

2 · INICIO
Sprint Planning
Elige 2 o 3 historias que alcanza a terminar en la semana.

CADA MAÑANA
Daily · 5 min
Anota qué hizo ayer, qué hará hoy y qué la bloquea.

9 · VIERNES
Sprint Review
Le muestra al cliente lo que funciona.

10 · AL CERRAR
Retrospectiva
Anota qué mejorar y actualiza su `AGENTS.md`.

Los números son los mismos del flujo de la página 4. El **Sprint** también es un evento: es el contenedor de una semana que termina con un incremento que funciona.

SUS ARTEFACTOS

Artefacto

Product Backlog

Lista priorizada de historias de usuario en un `BACKLOG.md` o en un tablero Kanban.

↓ en la planning salen las del sprint

Sprint Backlog · sprint 1

Artefacto

HU-01
Como cliente, quiero ver las horas disponibles de un día para elegir una.

HU-02
Como cliente, quiero reservar una hora indicando el nombre de mi mascota.

LA SEGUIMOS EN ESTE CASO

HU-03
Como recepcionista, quiero cancelar una reserva.

03 · EL FLUJO COMPLETO

De la historia al commit, y de vuelta

La HU-02 baja de Scrum a SDD, se construye y vuelve a Scrum. Usa los números como índice: las páginas siguientes y tu mapa (página 9) los repiten.

SCRUM · ORGANIZA EL TRABAJO

1. **Product Backlog** — Artefacto · historias priorizadas — pág. 3
2. **Sprint Planning** — Evento · elige 2 o 3 historias — pág. 3
3. **HU-02** — Reservar una hora — pág. 3

esta semana →

SDD · CONSTRUYE CADA HISTORIA

4. **spec.md** — Qué debe pasar y cómo se prueba — pág. 5
5. **plan.md · plan mode** — Claude propone; ella corrige y aprueba — pág. 6
6. **Tareas** — Cuatro pasos chicos — pág. 6
7. **Código + tests** — Un test por cada criterio — pág. 7
8. **Commit** — Recién después de leer el diff — pág. 7

← incremento funcionando

9. **Sprint Review** — Evento · el cliente ve el incremento — pág. 8
10. **Retrospectiva** — Evento · qué mejorar — pág. 8

→ **AGENTS.md** — Regla nueva que el agente leerá en la próxima historia — pág. 8

Paso de Scrum · Paso de SDD · Aquí trabaja Claude Code · La Daily acompaña todo el sprint

04 · PASO 4 DEL FLUJO · ESPECIFICAR

spec.md: primero el qué

Scrum le dijo a Valentina que esta semana toca la HU-02. Antes de abrir el editor, escribe **qué debe pasar** en `specs/HU-02/spec.md`.

SPECS/HU-02/SPEC.MD — MARKDOWN

```
# HU-02 — Reservar una hora

## Objetivo
El cliente reserva una hora libre indicando su
nombre, teléfono y el nombre de la mascota.

## Criterios de aceptación
- [ ] Solo se reservan horas libres; si la hora ya
      está tomada, se responde "Hora no
disponible".
- [ ] El teléfono debe tener 9 dígitos y empezar
      con 9.
- [ ] No se aceptan reservas en el pasado.
- [ ] Al reservar, la hora queda como "tomada".

## Fuera de alcance
Pagos online y recordatorios por correo.
```

Objetivo
Una sola frase: qué logra el cliente y con qué datos.

Criterios de aceptación
Cada uno es concreto y se puede comprobar. En el paso 7 cada criterio tendrá su propio test.

Fuera de alcance
Lo que **no** se construye ahora. Al escribirlo, nadie (ni Claude) lo agrega por su cuenta.

Sin una línea de código
La spec dice el **qué**. El **cómo** llega en el plan.

¿SE PUEDE PROBAR?

✗ CRITERIO FLOJO

`El teléfono debe ser válido.`

¿Válido según qué? Cada persona (y cada agente) lo interpretará distinto.

✓ CRITERIO DEL CASO

`El teléfono debe tener 9 dígitos y empezar con 9.`

Es fácil escribir un test que lo confirme o lo rechace.

05 · PASOS 5 Y 6 DEL FLUJO · PLANIFICAR Y DIVIDIR

Claude propone, Valentina decide

Valentina le pasa la spec a Claude Code en **plan mode**: Claude lee el proyecto y propone cómo hacerlo, pero no edita nada hasta que ella apruebe.

CLAUDE CODE

1 · Propone el plan

Lee la spec y el repo, y describe qué archivos tocaría y en qué orden.

→

VALENTINA

2 · Lo corrige

«Usa la tabla `reservas` que ya existe.»

→

VALENTINA

3 · Lo aprueba

Queda guardado como `plan.md` junto a la spec.

Tres formas de entrar a plan mode

`claude --permission-mode plan`

Al abrir Claude Code.

`Shift` + `Tab`

Durante la sesión, hasta que la barra de estado diga plan mode.

`/plan <tu pedido>`

Antes de un solo prompt.

En JetBrains funciona igual, desde la terminal del IDE. Verificado en code.claude.com/docs el 5 nov 2026.

PROMPTS PARA CLAUDE CODE — EJEMPLO

```
# 1. En plan mode
Lee specs/HU-02/spec.md y propón un
plan para implementarla.
No edites nada todavía.

# 2. Después de leer el plan
Usa la tabla reservas que ya existe.
Cuando apruebe el plan, guárdalo como
specs/HU-02/plan.md
```

Claude Code no guarda el plan como `plan.md` en tu repo por su cuenta: por eso Valentina se lo pide. Así el plan queda versionado y el próximo paso lo puede leer.

PASO 6 · DIVIDIR EN TAREAS

Pasos chicos que se pueden revisar uno a uno

TAREA 1

Validar los datos de entrada.

TAREA 2

Crear el endpoint `POST /reservas`.

TAREA 3

Escribir los tests de cada criterio.

TAREA 4

Hacer el formulario web.

06 · PASOS 7 Y 8 DEL FLUJO · IMPLEMENTAR Y VERIFICAR

Terminado no es «funciona en mi PC»

Claude Code implementa **tarea por tarea**. Cada criterio de la spec tiene su test, y Valentina revisa el diff antes de hacer commit.

	CRITERIO DE SPEC.MD	SU TEST
1	Solo se reservan horas libres; si está tomada: «Hora no disponible».	✓ en verde
2	El teléfono tiene 9 dígitos y empieza con 9.	✓ en verde
3	No se aceptan reservas en el pasado.	✓ en verde
4	Al reservar, la hora queda como «tomada».	✓ en verde



TERMINAL — EJEMPLO

```
git commit -m "feat: HU-02 reservar hora"
```

Definition of Done · HU-02

Cierra la historia

La HU-02 está «terminada» solo si cumple las tres:

- [ ] Todos los criterios de `spec.md` tienen un test en verde.
- [ ] Valentina leyó y entendió el diff.
- [ ] La spec refleja lo que realmente se construyó.

Nota: la Guía de Scrum define la Definition of Done para el incremento completo. Aquí Valentina la aplica a cada historia, algo habitual en equipos pequeños.

07 · PASOS 9 Y 10 DEL FLUJO · CERRAR EL CICLO

La semana termina; lo aprendido queda

Evento · viernes

Sprint Review

Valentina le muestra al cliente **lo que funciona**: el incremento de la semana. Patitas Felices ve el avance y opina antes del siguiente sprint.

Evento · al cerrar

Retrospectiva

Anota qué mejorar y lo deja escrito **donde el agente lo va a leer**: en su `AGENTS.md`.

AGENTS.MD · TURNOVET — EJEMPLO

```
# AGENTS.md
… reglas que ya tenía el proyecto …

## Aprendido en la retro del sprint 1
- Valida teléfonos chilenos con 9 dígitos.
```

Desde la próxima historia, Claude Code lee esta regla sin que Valentina tenga que repetirla en cada prompt (recuerda: lo hace solo si no hay un `CLAUDE.md`, como viste en tu guía del taller).

LO QUE NOS LLEVAMOS

Scrum sin SDD

Sprints ordenados, pero con código improvisado y difícil de revisar.

SDD sin Scrum

Especificaciones impecables, pero sin saber qué es lo prioritario para el cliente.

Los dos juntos

Scrum elige **la historia correcta** y SDD la construye **de la forma correcta**. Así trabaja la mayoría de las empresas de software, con más personas en cada rol.

PARA PENSAR

Si Valentina contratara a un segundo desarrollador mañana, ¿qué artefactos de este caso le servirían para incorporarse al proyecto el primer día?

08 · PARA EL PASO 4 DEL TALLER

Cómo usar este caso en tu mapa

En `docs/mapa-sdlc.md` completas una fila por etapa. Este caso no te da las respuestas: te muestra **dónde mirar** para pensar cada fila.

1 · PARA CADA COLUMNA, HAZLE ESTA PREGUNTA AL CASO

Etapa	Se delega a Claude	Artefacto versionado	Revisa una persona	Riesgo	Evento Scrum
¿De qué parte del ciclo hablas?	¿Qué le pidió Valentina a Claude en esos pasos?	¿Qué archivo quedó en el repo?	¿En qué momento decidió ella, y qué miró?	¿Qué habría salido mal si se saltaba ese paso?	¿En qué momento de la semana ocurrió?

2 · DÓNDE MIRAR PARA CADA FILA DE TU PLANTILLA

Requisitos	1 Product Backlog · 3 HU-02 · 4 spec.md	pág. 3 y 5
Diseño	5 plan.md en plan mode	pág. 6
Código	6 Tareas · 7 Implementación	pág. 6 y 7
Pruebas	7 Un test por criterio · Definition of Done	pág. 7
Revisión y despliegue	8 Diff y commit · 9 Sprint Review ⚠ El caso no muestra el despliegue: esa parte la piensas tú	pág. 7 y 8
Mejora continua	10 Retrospectiva · + AGENTS.md	pág. 8

⚠ **Tu mapa es de Pedidos Web, no de TurnoVet.** No copies filas: traduce la idea a tu proyecto y usa como evidencia lo que te pasó en el paso 3 (lo que la IA hizo mal y tú corregiste). Cada fila debe dejar claro dónde decide una persona.

09 · PARA APRENDER LO NECESARIO

Fuentes oficiales

Si quieres ir más allá del caso, empieza por aquí. Todos los enlaces también están en el sitio del curso.

SCRUM Y EL CICLO DE VIDA · PÁGINAS 3, 4 Y 8

Guía de Scrum
Roles, eventos y artefactos, incluida la Definition of Done.
`scrumguides.org`

The AI-native SDLC playbook (artículo)
Cada etapa deja un artefacto que la siguiente lee.
`claude.com/resources/articles/the-ai-native-sdlc-playbook`

SDD Y PLAN MODE · PÁGINAS 5 A 7

AI-native SDLC playbook · Introducción
El SDLC tradicional frente al AI-native.
`academy.claude.com/courses/ai-native-sdlc-playbook/introduction`

AI-native SDLC playbook · Plan mode
Cómo pedir, cuestionar y aprobar un plan.
`academy.claude.com/courses/ai-native-sdlc-playbook/plan-mode`

Claude Code · Modos de permiso
Cómo entrar y salir de plan mode. Consultado el 5 nov 2026.
`code.claude.com/docs/es/permission-modes`

Best practices for Claude Code
Explorar, planificar, programar y verificar.
`code.claude.com/docs/en/best-practices`

AGENTS.MD · PÁGINA 8

Estándar AGENTS.md
Qué va en el archivo y qué agentes lo leen.
`agents.md`

Claude Code · Memoria
Qué archivos de instrucciones lee Claude Code y cuándo.
`code.claude.com/docs/en/memory`

ESTA TARDE · UNIDAD 2

Desarrollo agéntico y SDD

Hoy viste a Valentina escribir su spec. En la tarde escribes la tuya y la llevas hasta un commit revisado.